$Space\ Complexity\ of\ Infix\ to\ Prefix\ conversion$

1. $Input\ Space\ Complexity$: $O(n)$

$The\ input\ space\ complexity\ refers\ strictly\ to\ the\ memory\ required$

$to\ store\ the\ input\ provided\ to\ the\ function.$

- $The\ function\ takes\ a\ pointer\ to\ a\ character\ array\ char\ *\ infix. Storing\ this\ original\ infix\ expression\ takes\ \boldsymbol{O(n)}\ space.$

2. $Auxiliary\ Space\ Complexity$: $O(n)$

$The\ auxiliary\ space\ complexity\ refers\ to\ the\ temporary, extra\ space$

$allocated\ by\ The\ algorithm\ to\ solve\ the\ problem, excluding\ the\ input\ itself.$

- $\boldsymbol{The\ Prefix\ Expression\ Array}\ (\boldsymbol{prefixExpr})$: $Dynamically\ allocate\ memory\ for\ the\ output\ string\ using\ malloc((len) * sizeof(char)).$
  $This\ requires\ \boldsymbol{O(n)}\ space.$

- $\boldsymbol{The\ Stack}\ (\boldsymbol{st})$: $Initializing\ the\ stack\ using\ create(\&st, len). In\ the\ worst-case\ scenario\ (e.g., an\ expression\ with\ many\ operators\ or\ unbalanced\ parentheses\ before\ popping), the\ stack\ will\ need\ to\ store\ up\ to\ n\ characters.$
  $Therefore, the\ stack's\ internal\ array\ also\ takes\ \boldsymbol{O(n)}\ space.$

- $\boldsymbol{Primitive\ Variables}$: $The\ variables\ len, i, j, start, end, and\ temp\ only\ take\ up\ a\ constant\ amount\ of\ space, which\ is\ \boldsymbol{O(1)}.$

$Since\ O(n) + O(n) + O(1)\ simplifies\ to\ O(n), The\ total\ auxiliary\ space\ is\ \boldsymbol{O(n)}.$

$Total\ Space\ Complexity$: $O(n)$

$When\ combining\ the\ input\ space\ and\ the\ auxiliary\ space, the\ overall$

$Space\ Complexity\ of\ The\ program\ evaluates\ to\ \boldsymbol{O(n)}.$